



Examen: Cafetería de Especialidad con JavaScript



Objetivo del Examen

En este examen práctico deberás desarrollar una **aplicación interactiva de gestión de pedidos para una cafetería de especialidad y pastelería artesanal**, conectando una interfaz frontend desarrollada con **HTML, CSS y JavaScript Vanilla** a un **servidor backend local en Node.js/Express**.

La aplicación debe permitir:

1. Consultar la carta de productos y las categorías desde el servidor backend local (`/api/productos` y `/api/categorias`).
2. Renderizar las tarjetas de la carta en `#catalogoGrid` con nombre, categoría, notas de cata, precio base e imagen HD, y poblar las opciones del selector `#filtroCategoria` .
3. Filtrar productos por categoría (`#filtroCategoria`) y realizar búsqueda de texto en tiempo real (`#inputBusqueda`).
4. Abrir el modal de comanda `#modalDetalle` al presionar "Ordenar Producto" en una tarjeta, calculando dinámicamente el precio total en `#precioTotalCalculado` según la cantidad de unidades (`precioBase * cantidad`).
5. Persistir las comandas en `localStorage` bajo la clave `'cafeteria_pedidos'` , renderizarlas en `#pedidosLista` y permitir limpiarlas con `#btnLimpiarPedidos` .



Tip para el examen: En `index.html` encontrarás bloques de código HTML comentados con las plantillas exactas de la tarjeta (`.producto-card`), del modal (`.detail-header-info`) y de la comanda registrada (`.pedido-card`). **Podés copiar y pegar estas plantillas directamente en tu código JavaScript** (usando template literals con comillas invertidas ``) para no perder tiempo escribiendo estructura HTML ni clases CSS.



Tabla de Entregas / Issues de GitHub

Cada entrega se corresponde con un **issue automático** en tu repositorio de GitHub. Para cerrar cada issue automáticamente, incluye el commit sugerido exacto al subir tu solución a la rama principal (`main`).

Entrega	Tarea a Realizar	Commit Sugerido
#1	Vincular <code>css/styles.css</code> y <code>js/script.js</code> en <code>index.html</code> .	<code>feat(html): vincular css y script js al html</code>
#2	Consumir la API local (<code>/api/productos</code> y <code>/api/categorias</code>) usando <code>fetch</code> y <code>async/await</code> .	<code>feat(js): consumir api de cafeteria con fetch y async await</code>
#3	Renderizar catálogo de productos y opciones en el DOM.	<code>feat(js): renderizar productos y opciones en el dom</code>
#4	Implementar filtrado, cálculo dinámico y modal de comanda.	<code>feat(js): implementar filtrado calculo y comanda</code>
#5	Persistir y gestionar comandas en <code>localStorage</code> .	<code>feat(js): persistir y gestionar comandas en localStorage</code>

Especificación Técnica y Requerimientos

1. Servidor Backend Local

El servidor Express provisto corre en el puerto `3000` con CORS habilitado:

- `GET http://localhost:3000/api/productos` : Devuelve la lista completa de productos de la carta.
- `GET http://localhost:3000/api/productos/:id` : Devuelve el detalle de un producto por su ID numérico.
- `GET http://localhost:3000/api/categorias` : Devuelve las categorías disponibles (`id` , `nombre`).

Para iniciar el servidor backend:

```
npm start
```

2. Elementos Clave del DOM

- `#filtroCategoria` : `<select>` para filtrar productos por categoría.
- `#inputBusqueda` : Input de texto para buscar por nombre o notas de cata en tiempo real.
- `#catalogoGrid` : Contenedor grid donde se renderizan las tarjetas de productos (`.producto-card`).
- `#modalDetalle` : Modal para visualizar el producto y registrar la comanda.
- `#detalleContent` : Contenedor dentro del modal donde se visualiza el producto seleccionado.
- `#formPedido` : Formulario de la comanda.
- `#cantidadProducto` : Input numérico para la cantidad de unidades.
- `#inputCliente` : Campo de texto para el nombre del cliente.
- `#inputNotas` : Campo de texto para notas o aclaraciones del pedido.
- `#precioTotalCalculado` : Elemento que muestra en tiempo real el valor final (`precioBase * cantidad`).
- `#btnConfirmarPedido` : Botón submit para confirmar la orden.
- `#pedidosLista` : Contenedor donde se listan las comandas generadas.
- `#btnLimpiarPedidos` : Botón para vaciar el historial de pedidos en `localStorage`.

- `.cerrar-modal`: Botones para cerrar el modal.

3. Lógica de Negocio y Cálculo Dinámico

- **Fórmula de Cálculo:** $\text{Total} = \text{Precio Base del Producto} \times \text{Cantidad}$
- Al modificar `#cantidadProducto` (`input` o `change`), `#precioTotalCalculado` debe actualizarse inmediatamente.
- Búsqueda en `#inputBusqueda` con `toLowerCase()` y filtrado interactivo combinado con el selector de categoría.
- Al enviar `#formPedido` (`submit`), prevenir recarga de página (`e.preventDefault()`), registrar la comanda con fecha/hora, persistirla en `localStorage` y cerrar el modal.

4. Almacenamiento Local (`localStorage`)

- **Clave obligatoria:** `'cafeteria_pedidos'`
- **Estructura:** Arreglo de objetos con `{ id, productoNombre, cliente, cantidad, notas, total, fecha }`.
- Utilizar `JSON.stringify()` para guardar y `JSON.parse()` para recuperar los datos.
- `#btnLimpiarPedidos` debe usar `localStorage.removeItem('cafeteria_pedidos')` o `localStorage.clear()`.

Comandos de Prueba y Autoevaluación

Antes de entregar, podés autoevaluar tu trabajo localmente:

```
# Ejecutar todas las pruebas automáticas
npm test

# Ejecutar una prueba individual
npm run test:link
npm run test:fetch
npm run test:render
npm run test:events
npm run test:storage

# Validar estilo y calidad de código
npm run lint
npm run format:check
```

Instrucciones para la Ejecución Local

1. Instalar dependencias:

```
npm install
```

2. Iniciar el servidor local:

```
npm start
```

3. Abrir `index.html` en el navegador (usando la extensión **Live Server** de VS Code).
4. Abrir la consola de herramientas de desarrollador (**F12**) para verificar peticiones de red y depurar posibles errores.